

(3) Scikit-Learn

<https://www.datacamp.com/cheat-sheet/scikit-learn-cheat-sheet-python-machine-learning>

calculate basic descriptive statistics of the income per capita.

```
df['IncomePerCap'].describe()
```

Distribution Tests

```
# does the data come from a normal distrbution?
diagnostic.kstest_normal(df['IncomePerCap'].values, dist = 'norm')

# (0.0637621121184162, 0.0009999999999998899)
# p_value < 0.05 -> we can reject the null hypothesis that the data comes
from a normal distribution!

#how about exponential?
diagnostic.kstest_normal(df['IncomePerCap'].values, dist = 'exp')
```

Sampling Data with pandas

```
#make 10 samples with replacement
sample1_counties = df.sample(n = 10, replace = True)

#make 10 samples without replacement
sample1_counties = df.sample(n = 10, replace = False)

#sometimes we want to sample in an ublanaced way, so that we upsample
datapoints of certain characteristic,
#and downsample the others. this can be acieved with weights parameter
#here we sample by upsampling counties with large population
sample2_counties = df.sample(n = 10, replace = False, weights =
df['TotalPop'])

#on average, the samples in the sample produce with reveighting now have
higher population, as we wanted!
print(sample1_counties['TotalPop'].mean())
print(sample2_counties['TotalPop'].mean())

# 488013.0
# 783979.1
```

Relationships between 2 variables

```
stats.pearsonr(df['IncomePerCap'],df['Employed'])
# (0.2646136320394489, 9.942215354237806e-53)
# There is a small (0.26), but significant (p < 0.05) positive
correlation.

stats.spearmanr(df['IncomePerCap'],df['Employed'])
# SpearmanrResult(correlation=0.30770631560595474,
pvalue=1.415296431173735e-71)

# Spearman rank correlation is also significant.
```

Is income per capita higher in New York counties compared to California counties?

```
df.loc[df['State'] == 'New York']['IncomePerCap'].mean()
df.loc[df['State'] == 'California']['IncomePerCap'].mean()

# 28189.75806451613
# 27902.603448275862
"""
We see that there is a ~300$ gap. Quite a lot!
But is it significantly higher? Let's use a t-test. This is a two-sided
test for the null hypothesis that the two independent samples have
identical average (expected) values.
"""

stats.ttest_ind(df.loc[df['State'] == 'New York']['IncomePerCap'],
df.loc[df['State'] == 'California']['IncomePerCap'])
# Ttest_indResult(statistic=0.19788117232375713,
pvalue=0.8434785239169611)

""" p is not smaller than 0.05 -> we cannot reject the null hypothesis
that the income is the same -> there is no significant difference
"""
```

Measure Uncertainty with bar plot

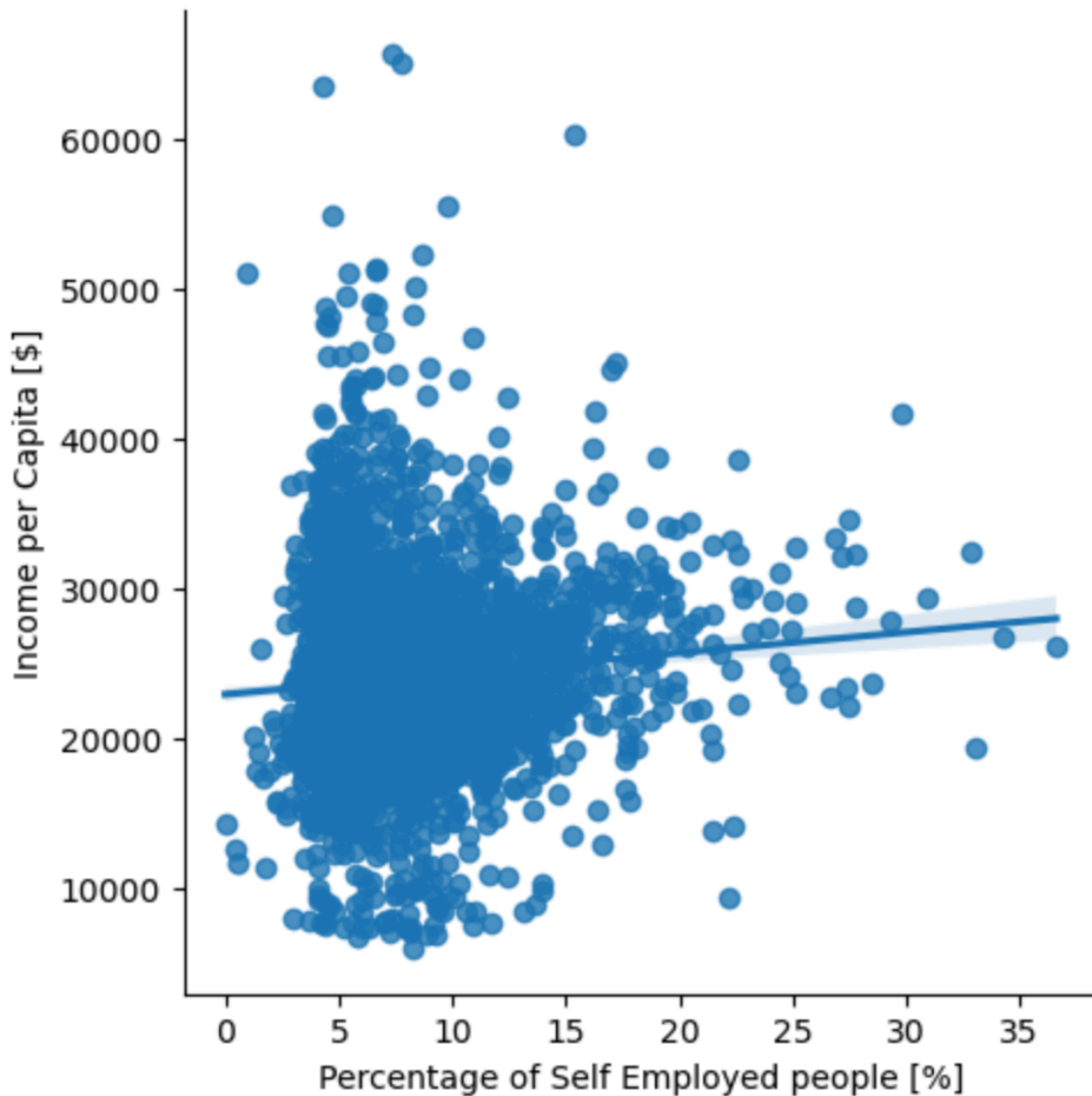
```
ax = sns.barplot(x="State", y="IncomePerCap",
data=df.loc[df['State'].isin(['New York','California'])])
plt.ylim([25000,32000])
```

Plot with CI

```
import seaborn as sn
```

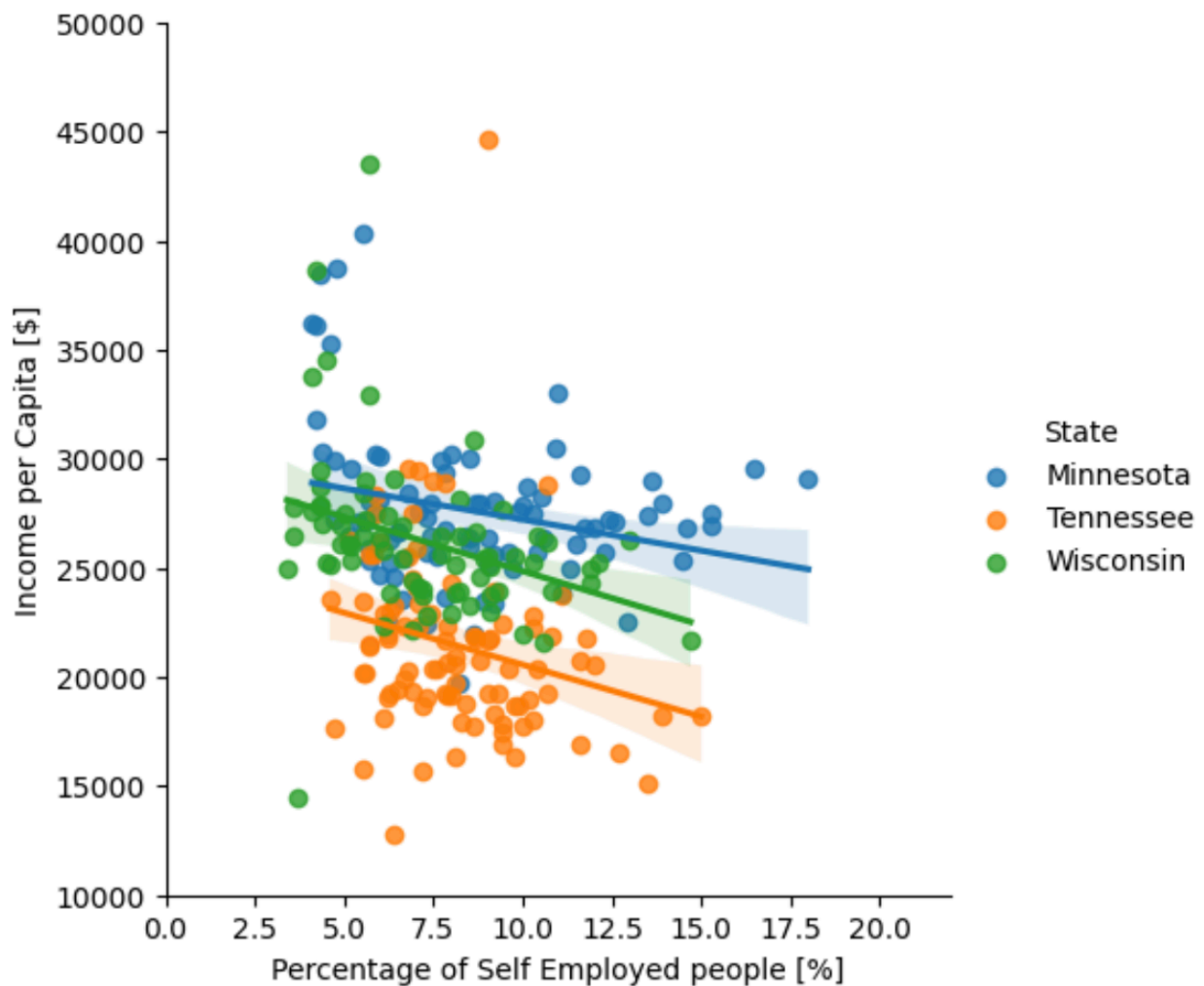
```
per_capita_self_empl = df[['State','IncomePerCap', 'SelfEmployed']]
sn.lmplot(x='SelfEmployed',y='IncomePerCap', data=per_capita_self_empl)
plt.xlabel("Percentage of Self Employed people [%]")
plt.ylabel("Income per Capita [$]")
```

```
Text(6.750000000000009, 0.5, 'Income per Capita [$]')
```



```
sn.lmplot(x='SelfEmployed',y='IncomePerCap',
data=SetA_per_capita_self_empl, hue = 'State')
plt.xlabel("Percentage of Self Employed people [%]")
plt.ylabel("Income per Capita [$]")
plt.ylim([10000,50000])
plt.xlim([0,22])
```

(0.0, 22.0)



```
Wisconsin_per_capita_self_empl = SetA_per_capita_self_empl.query("State ==  
'Wisconsin'")  
Tennessee_per_capita_self_empl = SetA_per_capita_self_empl.query("State ==  
'Tennessee'")  
Minnesota_per_capita_self_empl = SetA_per_capita_self_empl.query("State ==  
'Minnesota'")  
  
print(stats.pearsonr(Wisconsin_per_capita_self_empl['SelfEmployed'],Wisconsin_per_capita_self_empl['IncomePerCap']))  
print(stats.pearsonr(Tennessee_per_capita_self_empl['SelfEmployed'],Tennessee_per_capita_self_empl['IncomePerCap']))  
print(stats.pearsonr(Minnesota_per_capita_self_empl['SelfEmployed'],Minnesota_per_capita_self_empl['IncomePerCap']))  
  
#(-0.32905300016378525, 0.004768134887745222)  
#(-0.23836048684913147, 0.02001163195552807)  
#(-0.2538551921654062, 0.01766519930091192)
```

Some tst examples when needing to compare 2 variables

```

# =====
# STATISTICAL TESTS CHEAT SHEET (scipy.stats)
# =====
# Use this guide to choose the RIGHT test based on:
# - type of variables (numeric / categorical)
# - number of groups
# - independence / pairing
# - distribution assumptions
# =====

from scipy import stats

# -----
# 1. CORRELATION TESTS (association between TWO numeric variables)
# -----

# 1.1 Pearson correlation
# USE WHEN:
# - both variables are continuous
# - relationship is linear
# - no strong outliers
# - approximately normal
#
# TESTS:
# H0: no linear correlation
# H1: linear correlation exists
#
r, p = stats.pearsonr(x, y)

# -----

# 1.2 Spearman rank correlation
# USE WHEN:
# - variables are ordinal OR continuous
# - relationship is monotonic (not necessarily linear)
# - robust to outliers
#
# TESTS:
# H0: no monotonic association
# H1: monotonic association exists
#
rho, p = stats.spearmanr(x, y)

# -----

# 1.3 Kendall's Tau
# USE WHEN:
# - small sample size
# - many tied ranks

```

```

#
tau, p = stats.kendalltau(x, y)

# -----
# 2. COMPARING MEANS (numeric outcome)
# -----

# 2.1 One-sample t-test
# USE WHEN:
# - compare sample mean to a known value
#
# EXAMPLE: Is mean outcome different from 0?
#
t, p = stats.ttest_1samp(x, popmean=0)

# -----

# 2.2 Independent two-sample t-test (Welch's t-test)
# USE WHEN:
# - numeric outcome
# - two independent groups
# - do NOT assume equal variances
#
# EXAMPLE: male vs female outcomes
#
t, p = stats.ttest_ind(x1, x2, equal_var=False)

# -----

# 2.3 Paired t-test
# USE WHEN:
# - same subjects measured twice
# - before/after, pre/post
#
# EXAMPLE: outcomes before vs after training
#
t, p = stats.ttest_rel(before, after)

# -----
# 3. NON-PARAMETRIC ALTERNATIVES (no normality assumption)
# -----

# 3.1 Mann-Whitney U test
# USE WHEN:
# - two independent groups
# - outcome is ordinal or non-normal
#
# Alternative to independent t-test
#
u, p = stats.mannwhitneyu(x1, x2, alternative="two-sided")

```

```

# -----

# 3.2 Wilcoxon signed-rank test
# USE WHEN:
# - paired samples
# - non-normal differences
#
w, p = stats.wilcoxon(before, after)

# -----

# 3.3 Kruskal-Wallis test
# USE WHEN:
# - numeric outcome
# - more than two independent groups
# - non-normal
#
h, p = stats.kruskal(group1, group2, group3)

# -----
# 4. CATEGORICAL DATA TESTS
# -----

# 4.1 Chi-square test of independence
# USE WHEN:
# - two categorical variables
# - large enough expected counts
#
# EXAMPLE: gender vs success
#
chi2, p, dof, expected = stats.chi2_contingency(contingency_table)

# -----

# 4.2 Fisher's Exact Test
# USE WHEN:
# - two categorical variables
# - small sample sizes
# - 2x2 contingency table
#
odds_ratio, p = stats.fisher_exact(contingency_2x2)

# -----
# 5. VARIANCE TESTS
# -----

# 5.1 Levene's test
# USE WHEN:
# - test if group variances are equal

```

```

# - robust to non-normality
#
stat, p = stats.levene(group1, group2)

# -----

# 5.2 Bartlett's test
# USE WHEN:
# - test equal variances
# - data is normally distributed
#
stat, p = stats.bartlett(group1, group2)

# -----

# 6. NORMALITY TESTS
# -----

# 6.1 Shapiro-Wilk test
# USE WHEN:
# - check normality (small / medium samples)
#
stat, p = stats.shapiro(x)

# -----

# 6.2 Kolmogorov-Smirnov test
# USE WHEN:
# - compare distribution to known distribution
#
stat, p = stats.kstest(x, 'norm')

# -----

# 7. EFFECT SIZE (IMPORTANT FOR INTERPRETATION)
# -----

# Cohen's d (for mean differences)
import numpy as np

def cohens_d(x1, x2):
    return (np.mean(x1) - np.mean(x2)) / np.sqrt(
        (np.var(x1, ddof=1) + np.var(x2, ddof=1)) / 2
    )

# -----

# END OF CHEAT SHEET
# =====

```

